

Network Protocols

A Visual Guide — Diagrams of How TCP, UDP, HTTP(S), SSH, and DNS Work

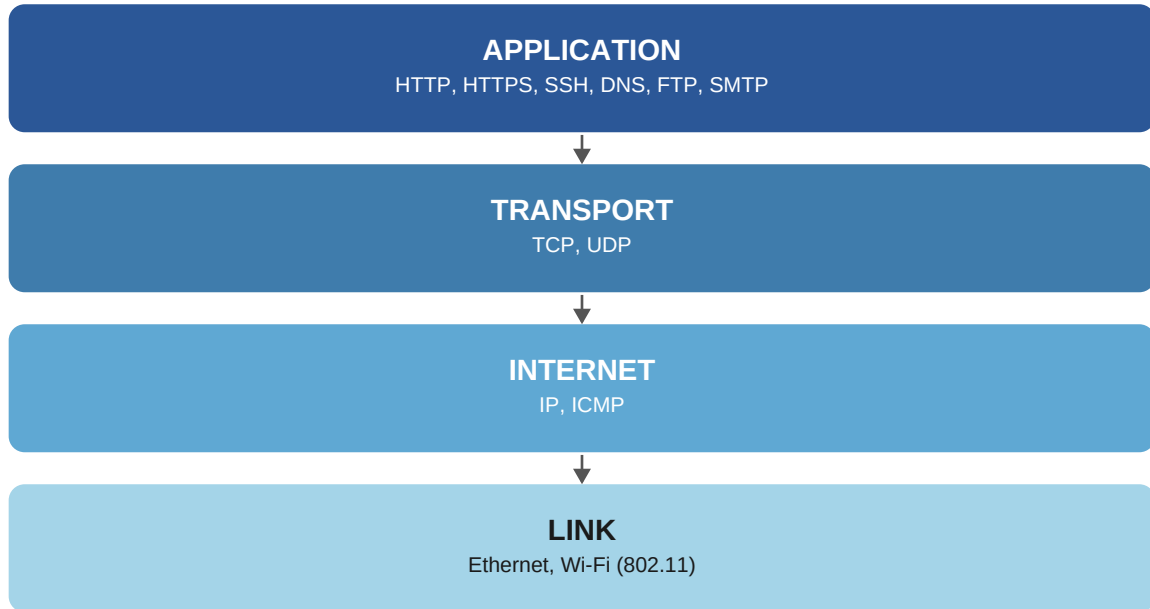
Every diagram in this guide is a vector graphic, drawn to scale, so it stays sharp at any zoom level.

Table of Contents

1. The Layered Network Stack
2. Encapsulation: How Data Gets Wrapped
3. TCP Three-Way Handshake and Teardown
4. TCP vs. UDP Side-by-Side
5. HTTP Request/Response Cycle
6. TLS Handshake (HTTPS)
7. SSH Tunnel / Port Forwarding
8. DNS Resolution Flow
9. One Host, Many Services (Port Map)

1. The Layered Network Stack

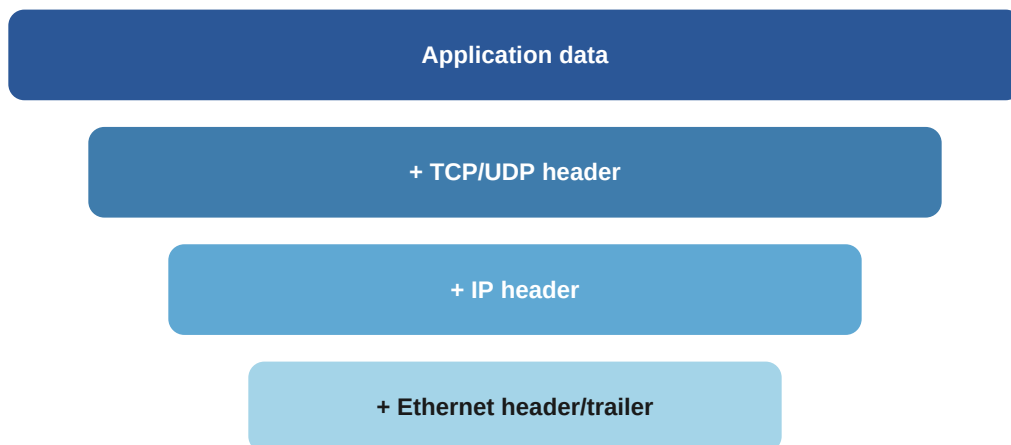
Data sent over a network passes down through layers on the sending side, and back up through the same layers on the receiving side. Each layer only needs to know how to talk to the layer directly above and below it.



Application-layer protocols ride on top of TCP or UDP, which ride on IP.

2. Encapsulation: How Data Gets Wrapped

As data travels down the stack, each layer adds its own header (and sometimes a trailer) around the data from the layer above — like nested envelopes.

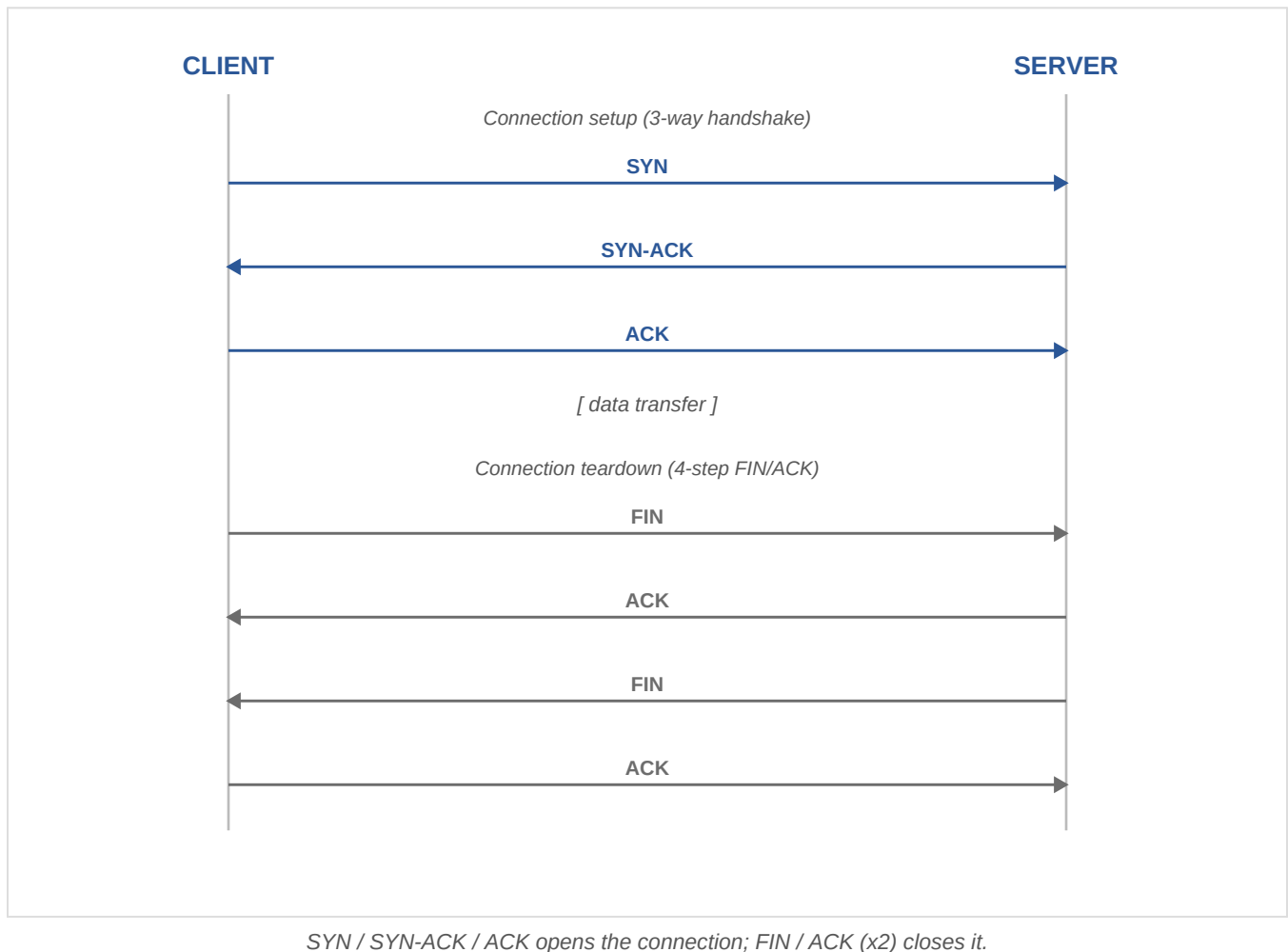


Each layer wraps the data from the layer above with its own header

The receiving side strips headers off in the reverse order.

3. TCP Three-Way Handshake and Teardown

TCP establishes a connection before sending data, and closes it explicitly when finished — this is what makes it reliable and ordered.



4. TCP vs. UDP Side-by-Side

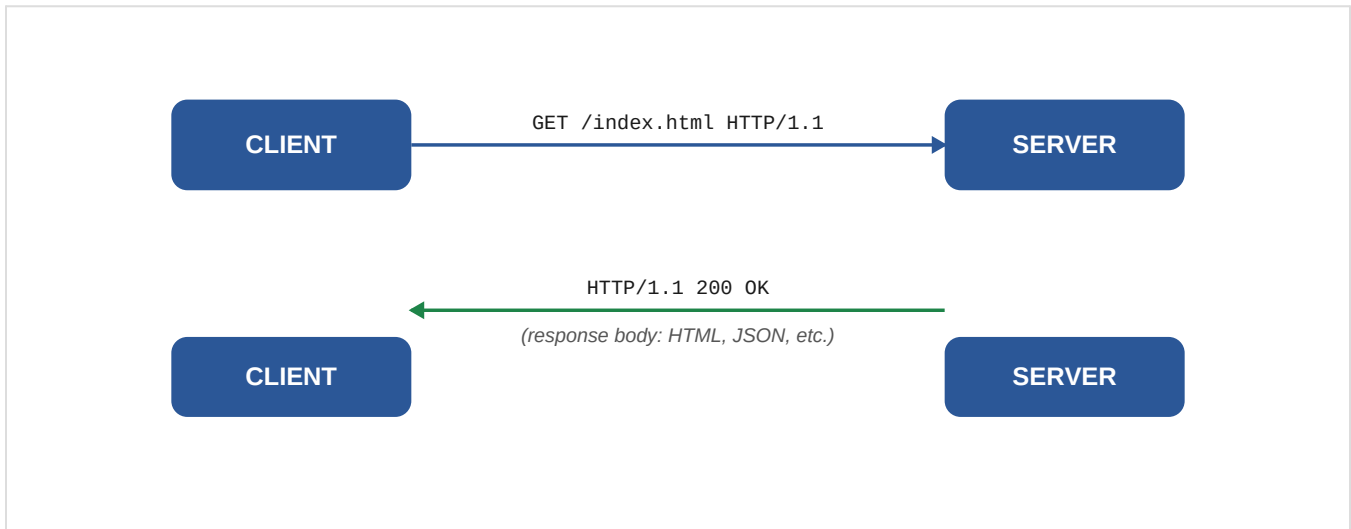
TCP trades speed for reliability; UDP trades reliability for speed. Neither is "better" — the right choice depends on the application.



TCP confirms and retransmits; UDP fires datagrams with no acknowledgment.

5. HTTP Request/Response Cycle

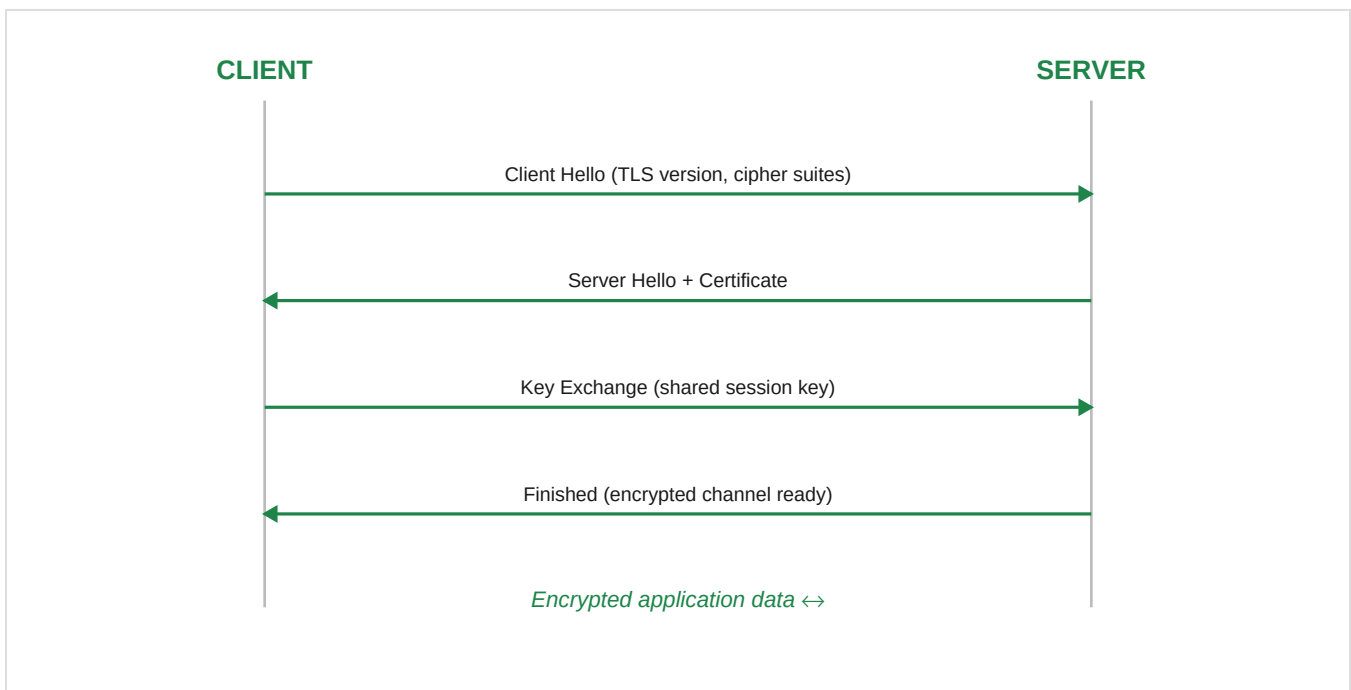
HTTP is a simple request-response protocol built on top of TCP: the client always initiates, and the server always responds.



Each HTTP request/response pair is independent (HTTP is stateless).

6. TLS Handshake (HTTPS)

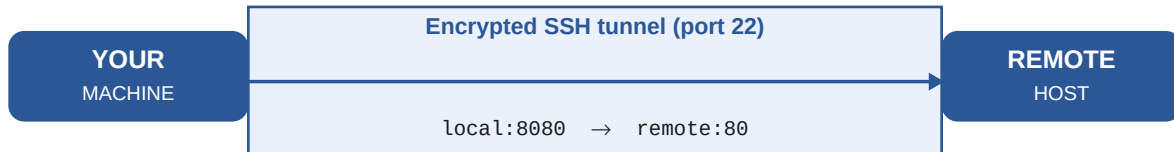
Before any encrypted HTTP data flows, the client and server negotiate encryption parameters and verify identity via the TLS handshake.



Once "Finished" is exchanged, all further data is encrypted.

7. SSH Tunnel / Port Forwarding

SSH can carry more than a terminal session — it can forward other TCP traffic through its encrypted channel, protecting protocols that aren't encrypted on their own.

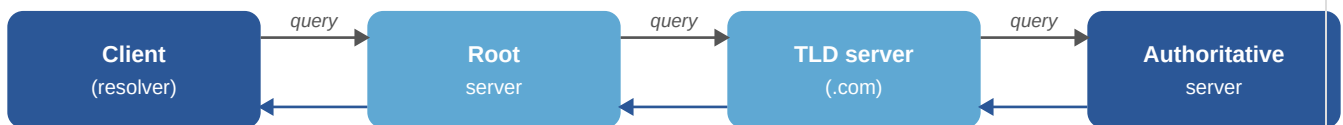


All forwarded traffic travels inside the encrypted SSH channel

Example: `ssh -L 8080:localhost:80 user@remote-host`

8. DNS Resolution Flow

Turning a domain name into an IP address usually involves several servers, each responsible for one part of the name.

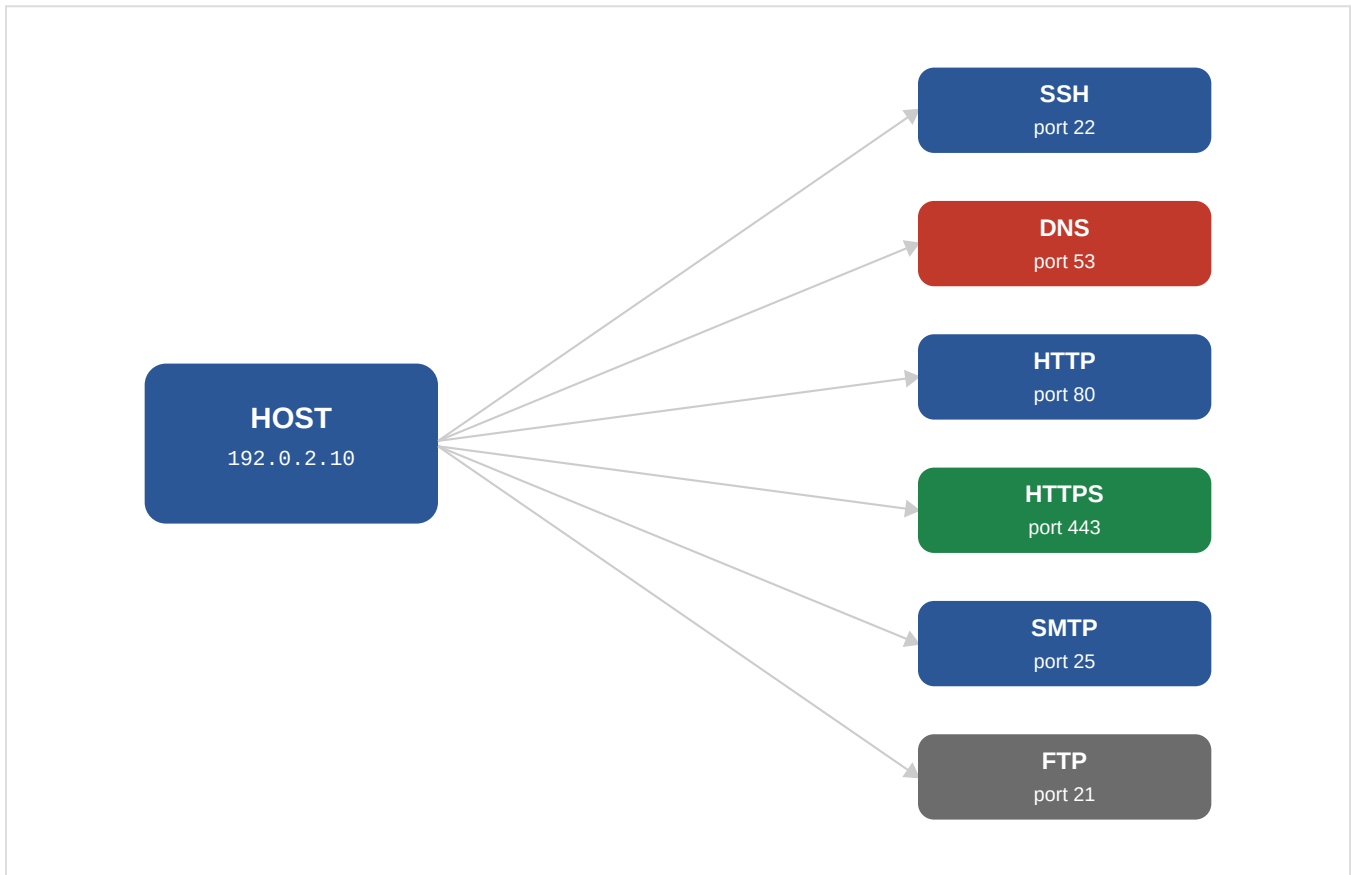


referral ↓ / answer ↑ (simplified — cached at each step)

Results are cached at each step, so most real-world lookups are much faster.

9. One Host, Many Services (Port Map)

A single machine can run many network services at once — the port number is what tells incoming traffic which service it's for.



The IP address identifies the host; the port identifies the service on it.